

Smart Pointers

Course: Rust Basics

Author: Aleksandr Kugushev

Null

```
struct User {  
    first_name: String,  
    second_name: String,  
    middle_name: String,  
}  
  
fn print(user: &User) {  
    println!("{}", user.first_name, user.second_name,  
user.middle_name);  
}
```

Null in Rust aka Option

```
let mut var: Option<i32> = Option::None;  
assert_eq!(true, var.is_none());  
var = Option::Some(42);  
assert_eq!(true, var.is_some());  
println!("{}", var.unwrap());  
let reference: &i32 = var.as_ref().unwrap();
```

Option in struct

```
struct User {  
    first_name: String,  
    second_name: String,  
    middle_name: Option<String>,  
}
```

Stack vs Heap

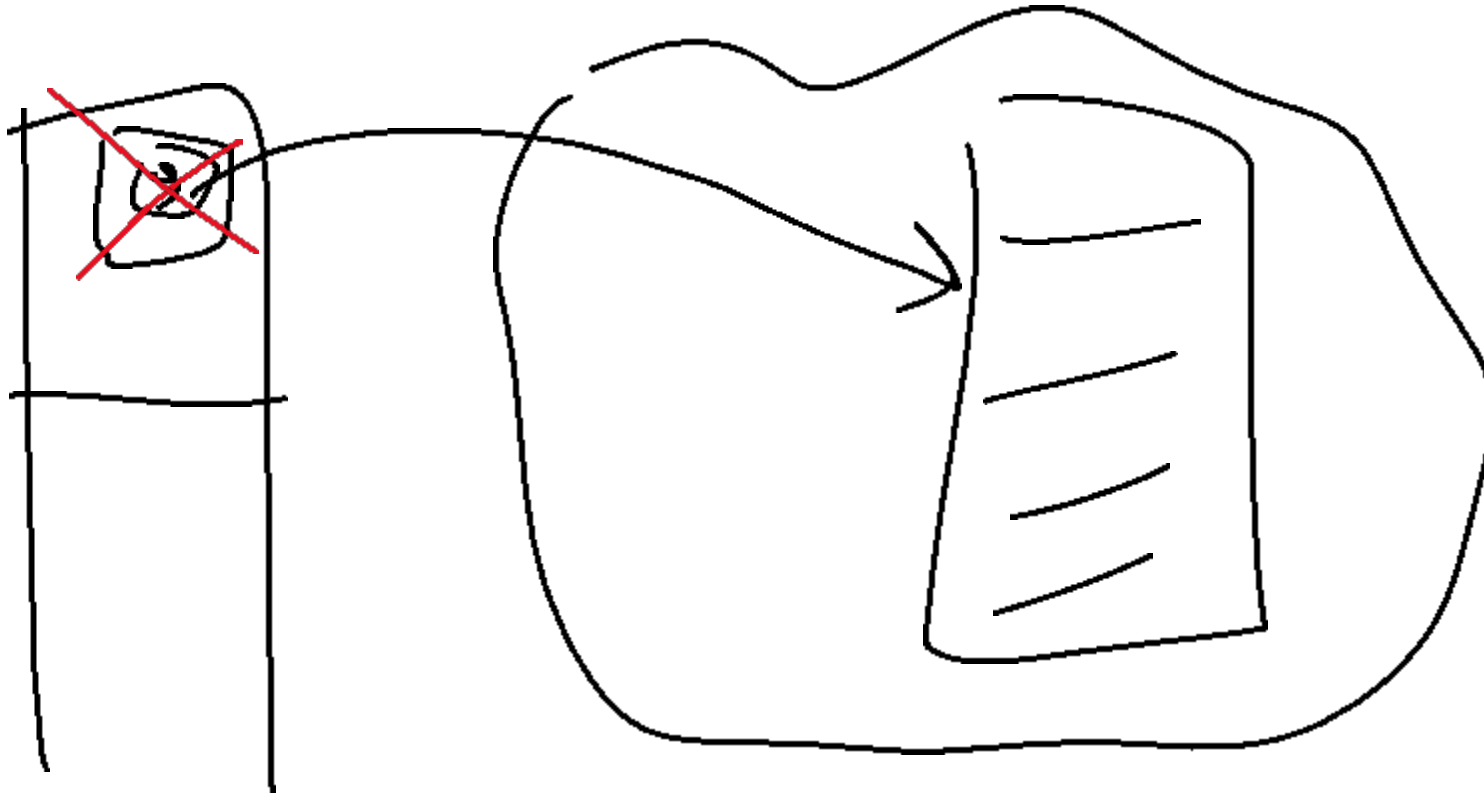
Let's draw

How to move data to heap?

```
unsafe {  
    let layout = Layout::new::<u16>();  
    let ptr = alloc(layout);  
    if ptr.is_null() {  
        handle_alloc_error(layout);  
    }  
  
    *(ptr as *mut u16) = 42;  
    assert_eq!(*(ptr as *mut u16), 42);  
  
    dealloc(ptr, layout);  
}
```

Problems?

Ownership Based Resource Management (OBRM)



MyObrm

```
struct MyObrm<T> {  
    ptr: *mut T,  
}
```

```
impl<T> MyObrm<T> {  
    fn new(val: T) -> Self {  
        unsafe {  
            let layout = Layout::new::<T>();  
            let ptr = alloc(layout) as *mut T;  
            if ptr.is_null() {  
                handle_alloc_error(layout);  
            }  
  
            *ptr = val;  
  
            MyObrm { ptr }  
        }  
    }  
}
```

MyObrm usage

```
fn obrm_usage() {  
    let user = MyObrm::new(MyStruct {  
        my_value: 42  
    });  
  
    println!("Print: {}", user.my_value);  
  
    drop(user);  
}
```

Deref and DerefMut traits

```
impl<T> Deref for MyObrm<T> {  
    type Target = T;  
    fn deref(&self) -> &Self::Target {  
        unsafe {  
            &*self.ptr  
        }  
    }  
}  
  
println!("Print: {}", user.my_value);
```

Drop trait

```
impl<T> Drop for MyObrm<T> {  
    fn drop(&mut self) {  
        unsafe {  
            let layout = Layout::new::<T>();  
            dealloc(self.ptr as *mut u8, layout);  
        }  
    }  
}
```

Box

- Former ~T syntax
- Now it is just a Rust structure Box<>

```
let smart_ptr: Box<i32> = Box::new(42);
```

```
struct Box<T>{  
    pointer: *const T  
}
```

Reference to Boxed value

```
let mut boxed = Box::new(MyStruct { my_value: 42 });  
let reference: &MyStruct = &boxed;  
println!("{}", reference.my_value);  
let mut_reference: &mut MyStruct = &mut boxed;  
mut_reference.my_value = 123;  
let reference: &Box<MyStruct> = &boxed;  
println!("{}", reference.my_value);
```

Box is special

```
let mut boxed = Box::new(MyStruct { my_value: 42 });  
let mut x: MyStruct = *boxed;  
x.my_value = 123;  
println!("{}", x.my_value);  
let reference = &x;  
println!("{}", boxed.my_value)  
error[E0382]: borrow of moved value: `*boxed`
```

Practice: Linked List



Linked List

```
struct Node{
    value: i32,
    next: Option<Box<Node>>
}
let node3 = Node{value: 3, next: None};
let node2 = Node{value: 2, next: Some(Box::new(node3))};
let node1 = Node{value: 1, next: Some(Box::new(node2))};

println!("{}", node1.value);
println!("{}", node1.next.unwrap().value);
println!("{}", node1.next.unwrap().next.unwrap().value);
^^^^^^^^ value used here after move
```

Unwrap

```
struct Node{  
    value: i32,  
    next: Option<Box<Node>>  
}
```

```
pub const fn unwrap(self)
```

Unwrap

```
struct Node{  
    value: i32,  
    next: Option<Box<Node>>  
}
```

```
pub const fn unwrap(self)
```

```
pub const fn as_ref(&self) -> Option<&T> {  
    match *self {  
        Some(ref x) => Some(x),  
        None => None,  
    }  
}
```

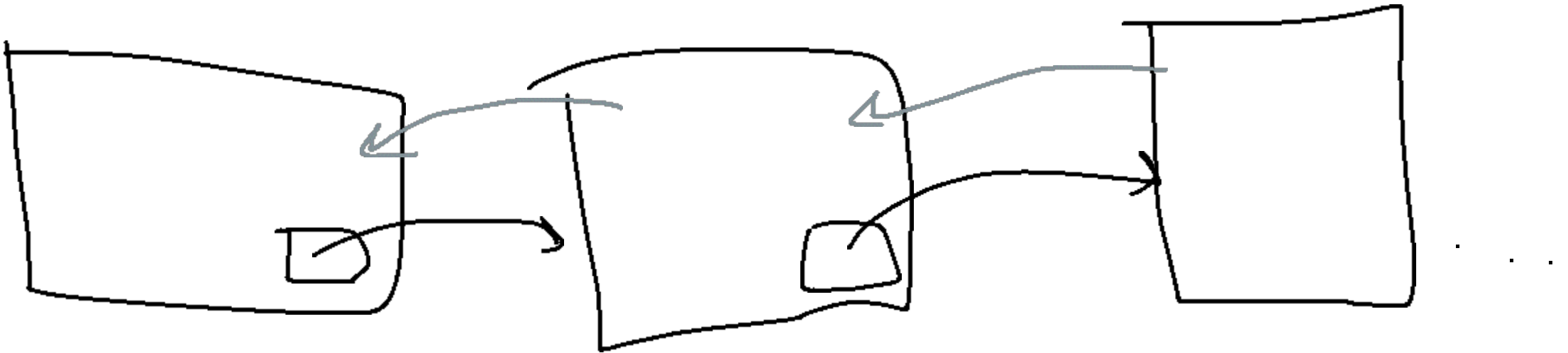
Linked List

```
struct Node{
    value: i32,
    next: Option<Box<Node>>
}

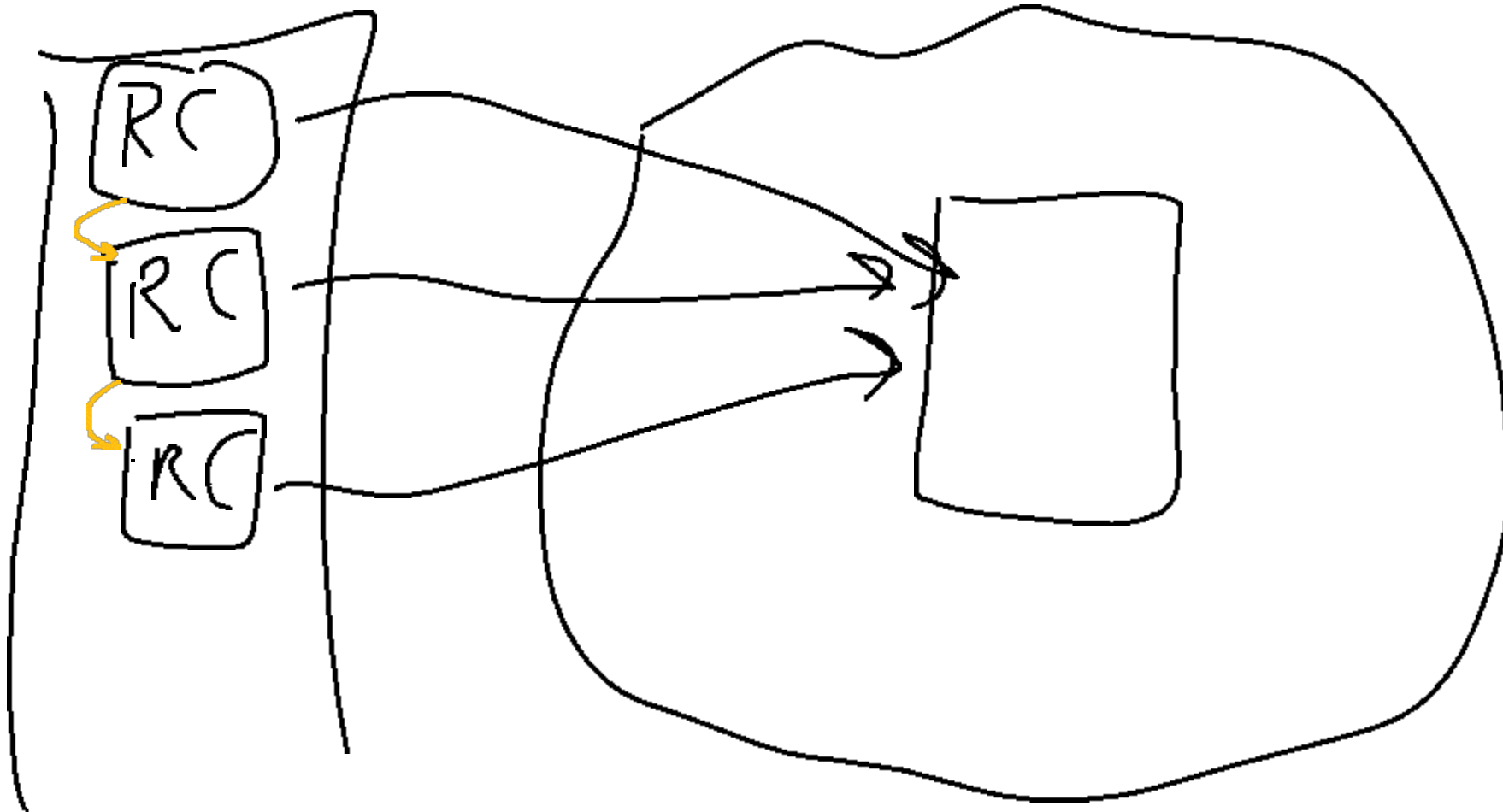
let node3 = Node{value: 3, next: None};
let node2 = Node{value: 2, next: Some(Box::new(node3))};
let node1 = Node{value: 1, next: Some(Box::new(node2))};

println!("{}", node1.value);
let node1_next: Option<&Box<Node>> = node1.next.as_ref();
println!("{}", node1_next.unwrap().value);
println!("{}", node1.next.as_ref().unwrap().next.as_ref().unwrap().value);
```

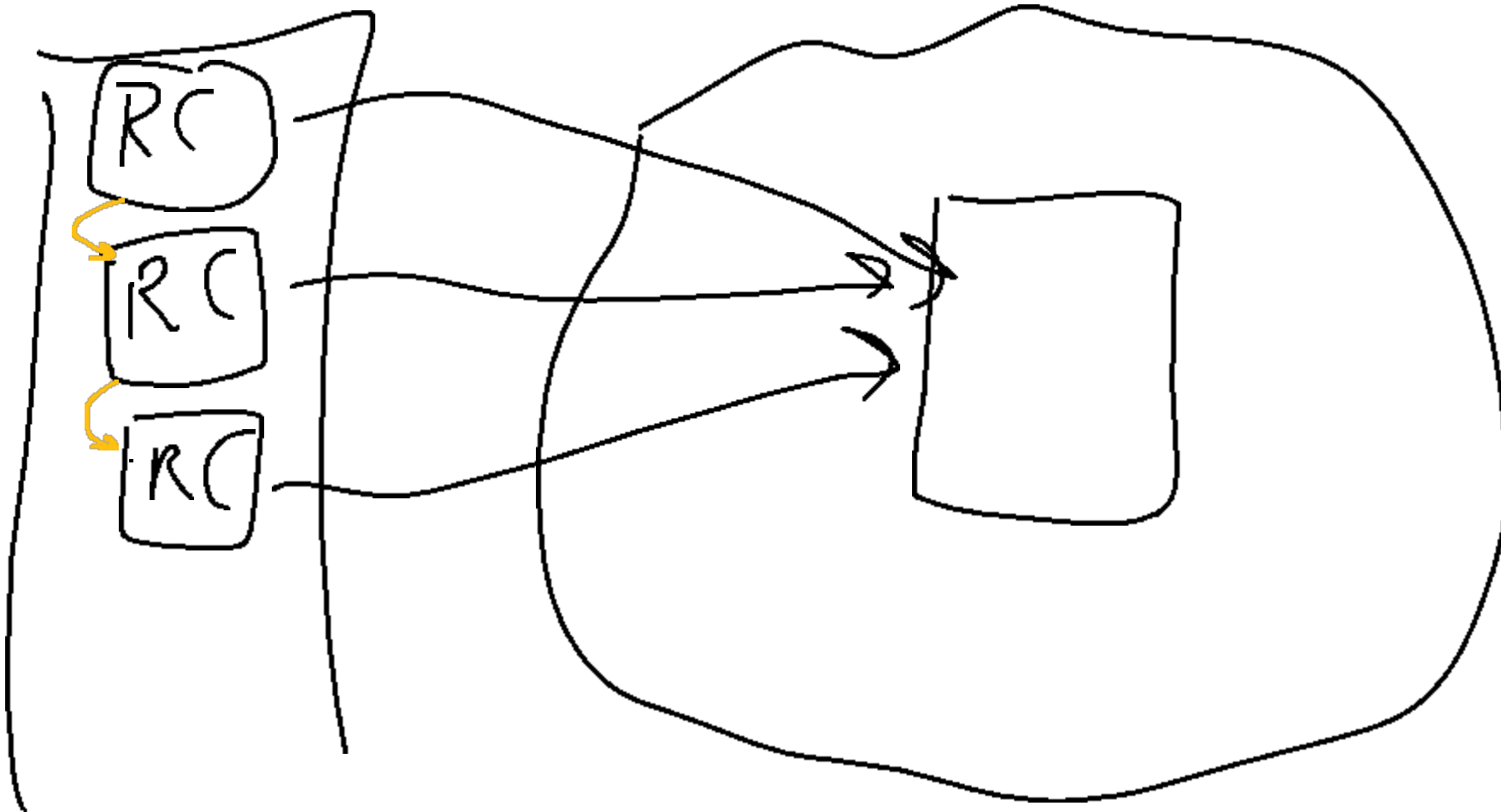
Practice: Previous in linked list



RC – shared ownership

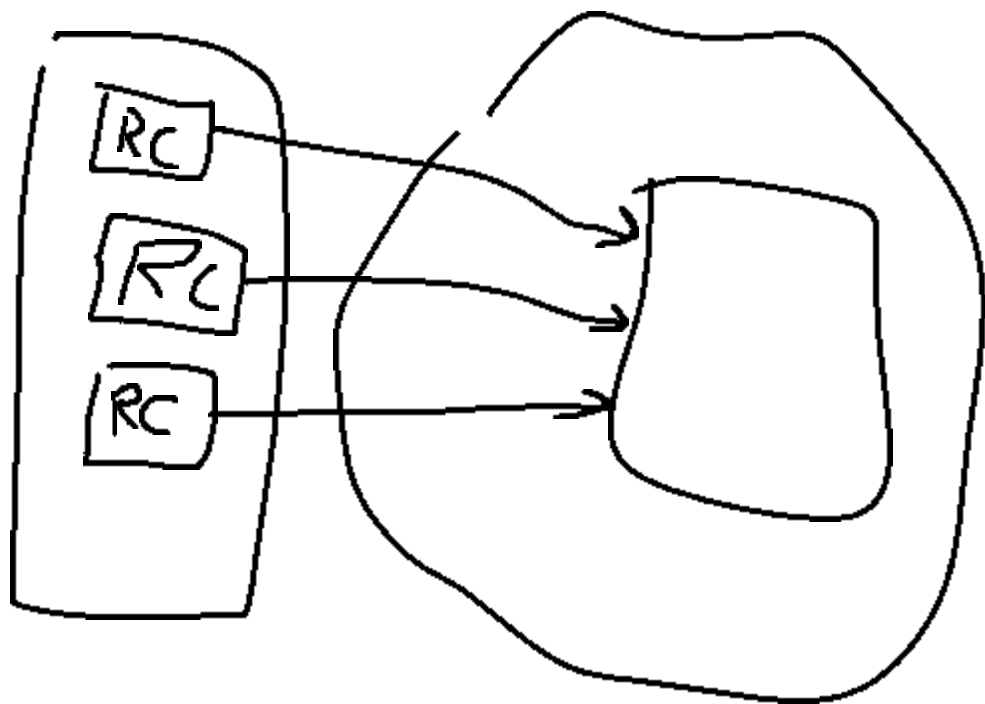


RC – shared ownership

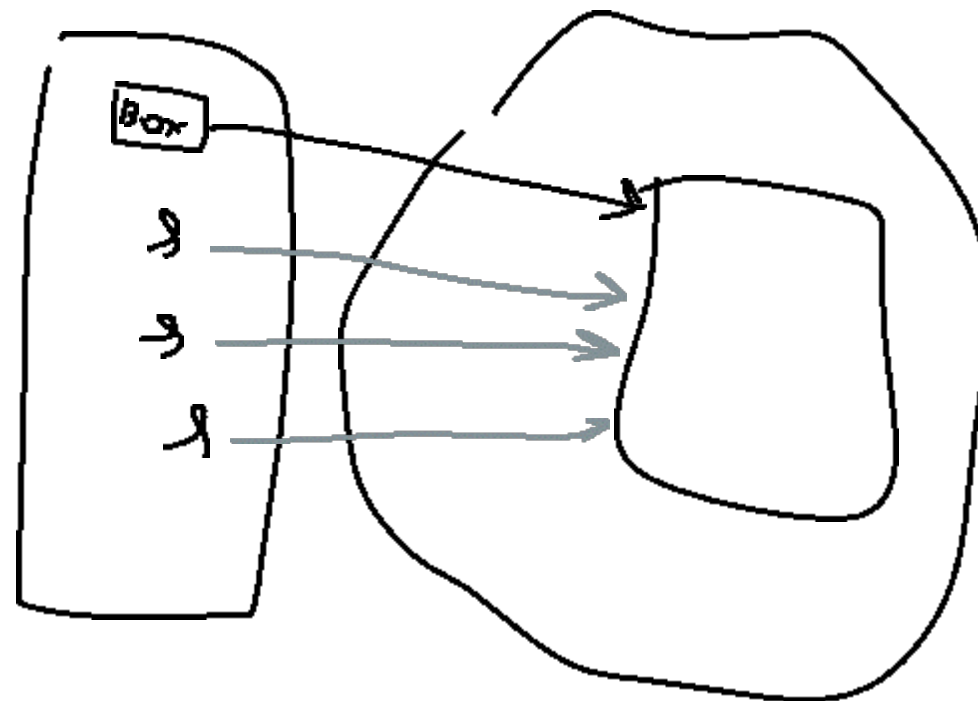


RC vs Box

RC



&Box



Rc

```
let owner1 = Rc::new(MyStruct { my_value: 42 });  
let owner2: Rc<MyStruct> = owner1.clone();  
let owner3: Rc<MyStruct> = owner2.clone();  
println!("{}", owner3.my_value);  
println!("{}", owner1.my_value);  
println!("{}", owner2.my_value);  
let reference: &MyStruct = &owner1;  
println!("{}", reference.my_value);
```

Edit Rc value

```
let owner1 = Rc::new(MyStruct { value1: 42, value2: true });  
owner1.value1 = 123;
```

error[E0594]: cannot assign to data in an `Rc`

Interior Mutability

```
let cell = Cell::new(42);  
let reference1: &Cell<i32> = &cell;  
let reference2: &Cell<i32> = &cell;  
println!("{}", reference1.get()); // > 42  
reference1.replace(123);  
println!("{}", reference2.get()); // > 123
```

Cell – Alias Safe

```
pub fn replace(&self, val: T) -> T {
    mem::replace(unsafe { &mut *self.value.get() }, val) // wrapped assignment
}

pub const fn replace<T>(dest: &mut T, src: T) -> T {
    // SAFETY: We read from `dest` but directly write `src` into it afterwards,
    // such that the old value is not duplicated. Nothing is dropped and
    // nothing here can panic.
    unsafe {
        let result = ptr::read(dest);
        ptr::write(dest, src);
        result
    }
}
```

How to get data from Cell

- `get()` – returns **copy!**
- `take()` – return value and replace with Default
- `replace()` – return value and replace

RefCell

```
let cell = RefCell::new(MyStruct { my_value: 42 });
{
    let reference1: Ref<MyStruct> = cell.borrow();
    println!("{}", reference1.my_value); // > 42
}
{
    let mut borrowed: RefMut<MyStruct> = cell.borrow_mut();
    borrowed.my_value = 123;
}

let reference2: Ref<MyStruct> = cell.borrow();
println!("{}", reference2.my_value); // > 123
```

RefCell

```
let cell = RefCell::new(MyStruct { my_value: 42 });  
                                Ref != &
```

```
let reference1: Ref<MyStruct> = cell.borrow();  
println!("{}", reference1.my_value); // > 42
```

PANIC: already borrowed: BorrowMutError

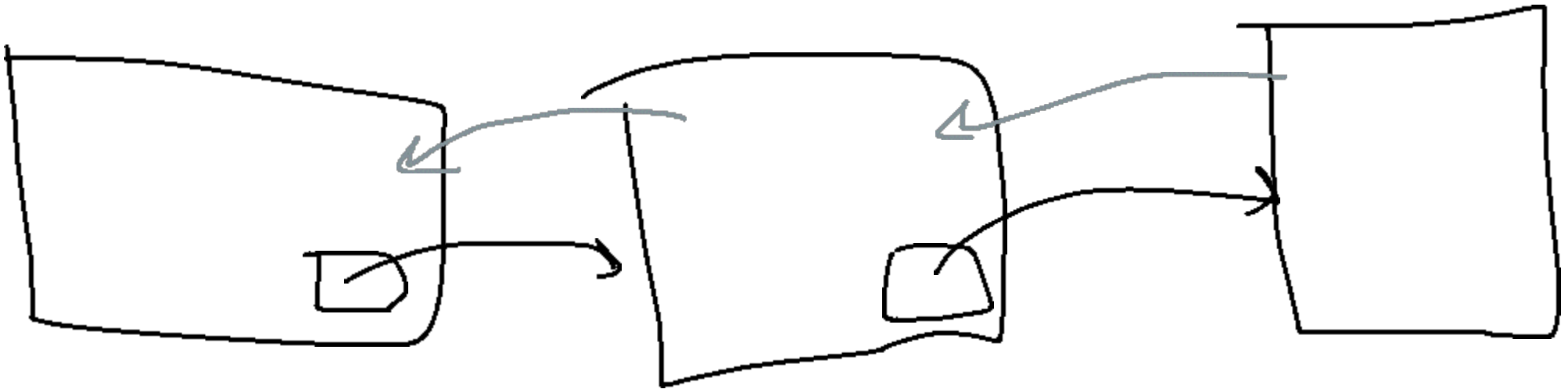
```
{  
    let mut borrowed: RefMut<MyStruct> = cell.borrow_mut();  
    borrowed.my_value = 123;  
}
```

```
let reference2: Ref<MyStruct> = cell.borrow();  
println!("{}", reference2.my_value); // > 123
```

RefMut

```
pub struct RefMut<'b, T: ?Sized + 'b> {  
    value: NonNull<T>,  
    borrow: BorrowRefMut<'b>,  
    marker: PhantomData<&'b mut T>,  
}  
  
fn unwrap_left_mut(&mut self) -> RefMut<'_, Node>
```

Practice: Previous in linked list



Circular references

```
struct NodeLeft {  
    right: Cell<Option<Rc<NodeRight>>>,  
}
```

```
struct NodeRight {  
    left: Cell<Option<Rc<NodeLeft>>>,  
}
```

```
let node_left = Rc::new(NodeLeft { right: Cell::new(None) });
```

```
let node_right = Rc::new(NodeRight {  
    left: Cell::new(Some(node_left.clone()))  
});
```

```
let e = node_left.right.replace(Some(node_right.clone()));
```

Rc::downgrade()

```
struct NodeLeft {  
    right: Cell<Option<Rc<NodeRight>>>,  
}  
  
struct NodeRight {  
    left: Cell<Option<Weak<NodeLeft>>>,  
}  
  
let node_left = Rc::new(NodeLeft { right: Cell::new(None) });  
let node_right = Rc::new(NodeRight {  
    left: Cell::new(Some(Rc::downgrade(&node_left)))  
});  
let e = node_left.right.replace(Some(node_right.clone()));
```

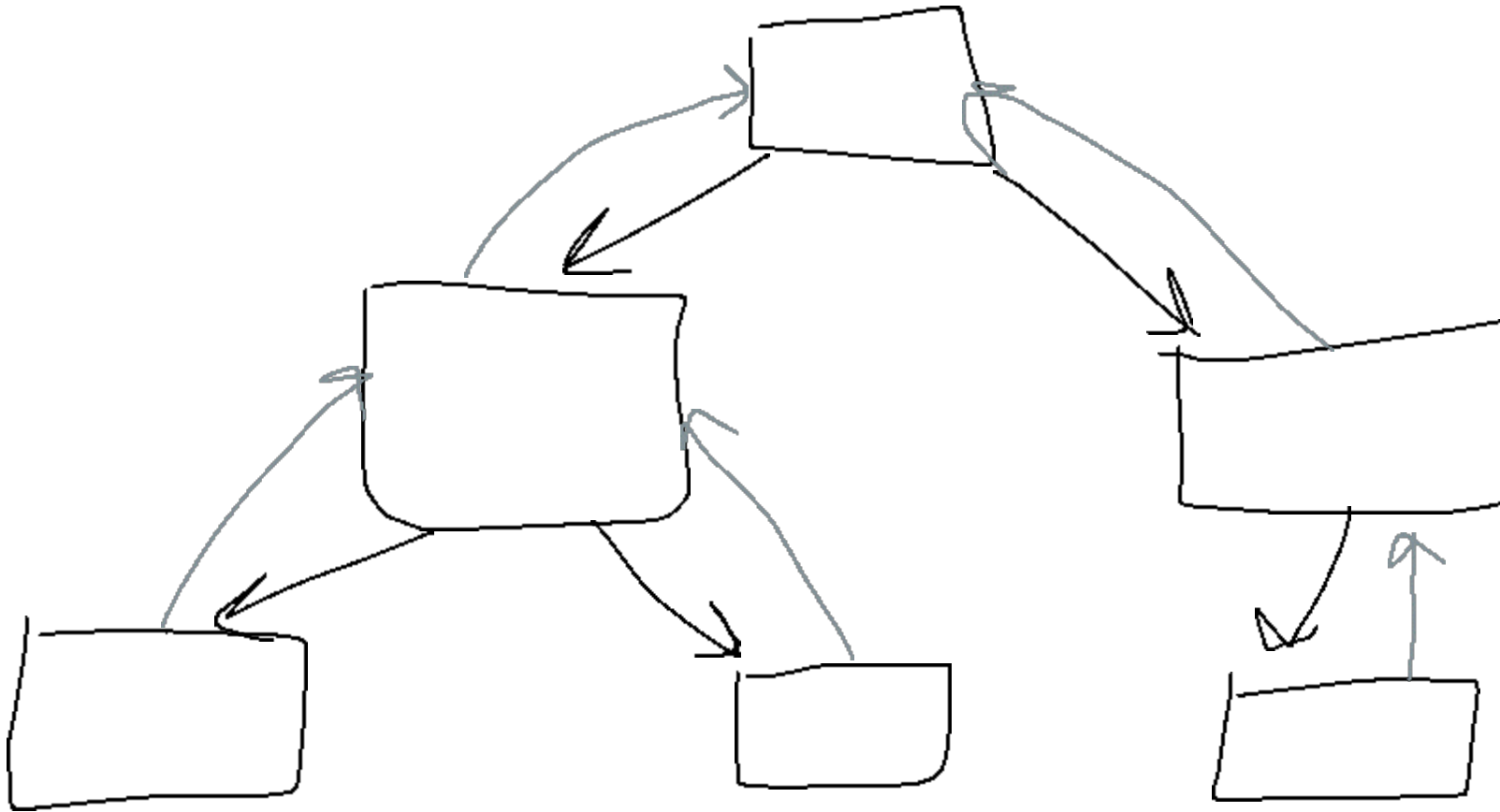
upgrade()

```
let five = Rc::new(5);  
let weak_five = Rc::downgrade(&five);  
let strong_five: Option<Rc<_>> = weak_five.upgrade();  
assert!(strong_five.is_some());  
drop(strong_five);  
drop(five);  
assert!(weak_five.upgrade().is_none());
```

Why `Rc::downgrade()`, but not
`rc1. downgrade()`?

Deref

Practice: Tree



Vec

```
pub struct Vec<T> {  
    pointer: *const T,  
    len: usize,  
}
```

```
let mut vec: Vec<i32> = Vec::new();  
vec.push(42);  
let last: Option<i32> = vec.pop();
```

Vec

```
let mut vec = vec![1, 2, 3];
```

```
let count = vec.len();
```

```
let first:i32 = vec[0];
```

```
let mut vec = vec![String::from("Hi")];
```

```
let first: String = vec[0];
```

error[E0507]: cannot move out of index of `Vec<String>`

Get by reference

```
let mut vec = vec![String::from("Hi")];
```

```
let first: &String = &vec[0];
```

```
let first: Option<&String> = vec.get(0);
```

Borrow element, borrow whole vector

```
let mut vec = vec![String::from("Hi")];
```

```
let first: &String = &vec[0];
```

```
vec.push(String::from("World"));
```

```
println!("{}", first);
```

error[E0502]: cannot borrow `vec` as mutable because it is also borrowed as immutable

Set vec element

```
let first_mut: &mut String = &mut vec[0];  
*first_mut = String::from("Hello");
```

```
vec[0] = String::from("World");
```

Internals

Box internals

```
pub struct Box<T: ?Sized, A: Allocator = Global>(Unique<T>, A);
```

```
pub struct Unique<T: ?Sized> {  
    pointer: NonNull<T>,  
    // NOTE: this marker has no consequences for variance, but is necessary  
    // for dropck to understand that we logically own a `T`.  
    _marker: PhantomData<T>,  
}
```

```
#[lang = "phantom_data"]
```

```
pub struct PhantomData<T: ?Sized>;
```

NonNull

```
pub struct NonNull<T: ?Sized> {  
    pointer: *const T,  
}
```

Rc internals

```
pub struct Rc<T: ?Sized, A: Allocator = Global> {  
    ptr: NonNull<RcBox<T>>,  
    phantom: PhantomData<RcBox<T>>,  
    alloc: A,  
}  
  
struct RcBox<T: ?Sized> {  
    strong: Cell<usize>,  
    weak: Cell<usize>,  
    value: T,  
}
```

Rc::clone()

```
fn clone(&self) -> Self {  
    unsafe {  
        self.inner().inc_strong();  
        Self::from_inner_in(self.ptr, self.alloc.clone())  
    }  
}  
  
fn inner(&self) -> &RcBox<T> {  
    // This unsafety is ok because while this Rc is alive we're  
guaranteed  
// that the inner pointer is valid.  
    unsafe { self.ptr.as_ref() }  
}
```

Rc::drop()

```
unsafe impl<T: ?Sized, A: Allocator> Drop for Rc<T, A> {
    fn drop(&mut self) {
        unsafe {
            self.inner().dec_strong();
            if self.inner().strong() == 0 {
                // destroy the contained object
                ptr::drop_in_place(Self::get_mut_unchecked(self));

                // remove the implicit "strong weak" pointer now that we've
                // destroyed the contents.
                self.inner().dec_weak();

                if self.inner().weak() == 0 {
                    self.alloc.deallocate(self.ptr.cast(), Layout::for_value(self.ptr.as_ref()));
                }
            }
        }
    }
}
```

upgrade()

```
pub fn upgrade(&self) -> Option<Rc<T, A>>
where
    A: Clone,
{
    let inner = self.inner()?;

    if inner.strong() == 0 {
        None
    } else {
        unsafe {
            inner.inc_strong();
            Some(Rc::from_inner_in(self.ptr, self.alloc.clone()))
        }
    }
}
```

Cell internals

```
pub struct Cell<T: ?Sized> {  
    value: UnsafeCell<T>,  
}  
  
pub struct UnsafeCell<T: ?Sized> {  
    value: T,  
}
```

Cell get()

Cell

```
impl<T: Copy> Cell<T> {  
    pub fn get(&self) -> T {  
        unsafe { *self.value.get() }  
    }  
}
```

UnsafeCell

```
impl<T: ?Sized> UnsafeCell<T> {  
    pub const fn get(&self) -> *mut T {  
        self as *const UnsafeCell<T>  
        as *const T as *mut T  
    }  
}
```

Cell replace()

```
pub fn replace(&self, val: T) -> T {  
    mem::replace(unsafe { &mut *self.value.get() }, val)  
}
```

```
pub const fn replace<T>(dest: &mut T, src: T) -> T {  
    unsafe {  
        let result = ptr::read(dest);  
        ptr::write(dest, src);  
        result  
    }  
}
```

RefCell internals

```
struct RefCell<T: ?Sized> {  
    borrow: Cell<BorrowFlag>,  
    borrowed_at: Cell<Option<&'static  
crate::panic::Location<'static>>>,&br/>    value: UnsafeCell<T>,  
}  
type BorrowFlag = isize;
```

borrow()

```
pub fn borrow(&self) -> Ref<'_, T> {  
    self.try_borrow().expect("already mutably borrowed")  
}
```

try_borrow()

```
pub fn try_borrow(&self) -> Result<Ref<'_, T>, BorrowError> {
    match BorrowRef::new(&self.borrow) {
        Some(b) => {
            {
                // `borrowed_at` is always the *first* active borrow
                if b.borrow.get() == 1 {
                    self.borrowed_at.set(Some(crate::panic::Location::caller()));
                }
            }

            // SAFETY: `BorrowRef` ensures that there is only immutable access
            // to the value while borrowed.
            let value = unsafe { NonNull::new_unchecked(self.value.get()) };
            Ok(Ref { value, borrow: b })
        }
        None => Err(BorrowError {
            // If a borrow occurred, then we must already have an outstanding borrow,
            // so `borrowed_at` will be `Some`
            location: self.borrowed_at.get().unwrap(),
        }),
    }
}
```

BorrowRef

```
impl<'b> BorrowRef<'b> {  
    fn new(borrow: &'b Cell<BorrowFlag>) -> Option<BorrowRef<'b>> {  
        let b = borrow.get().wrapping_add(1);  
        if !is_reading(b) {  
            // Incrementing borrow can result in a non-reading value (<= 0) in these cases:  
            // 1. It was < 0, i.e. there are writing borrows, so we can't allow a read borrow  
            //    due to Rust's reference aliasing rules  
            // 2. It was isize::MAX (the max amount of reading borrows) and it overflowed  
            //    into isize::MIN (the max amount of writing borrows) so we can't allow  
            //    an additional read borrow because isize can't represent so many read borrows  
            //    (this can only happen if you mem::forget more than a small constant amount of  
            //    Ref's, which is not good practice)  
            None  
        } else {  
            // Incrementing borrow can result in a reading value (> 0) in these cases:  
            // 1. It was = 0, i.e. it wasn't borrowed, and we are taking the first read borrow  
            // 2. It was > 0 and < isize::MAX, i.e. there were read borrows, and isize  
            //    is large enough to represent having one more read borrow  
            borrow.set(b);  
            Some(BorrowRef { borrow })  
        }  
    }  
}  
  
fn is_reading(x: BorrowFlag) -> bool {  
    x > UNUSED  
}  
  
const UNUSED: BorrowFlag = 0;
```

Ref

```
pub struct Ref<'b, T: ?Sized + 'b> {  
    value: NonNull<T>,  
    borrow: BorrowRef<'b>,  
}  
  
pub struct NonNull<T: ?Sized> {  
    pointer: *const T,  
}  
  
struct BorrowRef<'b> {  
    borrow: &'b Cell<BorrowFlag>,  
}  
  
type BorrowFlag = isize;
```

try_borrow_mut()

```
pub fn try_borrow_mut(&self) -> Result<RefMut<'_, T>, BorrowMutError> {  
    match BorrowRefMut::new(&self.borrow) {  
        Some(b) => {  
            {  
                self.borrowed_at.set(Some(crate::panic::Location::caller()));  
            }  
  
            // SAFETY: `BorrowRefMut` guarantees unique access.  
            let value = unsafe { NonNull::new_unchecked(self.value.get()) };  
            Ok(RefMut { value, borrow: b, marker: PhantomData })  
        }  
        None => Err(BorrowMutError {  
            // If a borrow occurred, then we must already have an outstanding borrow,  
            // so `borrowed_at` will be `Some`  
            location: self.borrowed_at.get().unwrap(),  
        }),  
    }  
}
```

BorrowRefMut

```
impl<'b> BorrowRefMut<'b> {  
  fn new(borrow: &'b Cell<BorrowFlag>) -> Option<BorrowRefMut<'b>> {  
    // NOTE: Unlike BorrowRefMut::clone, new is called to create the initial  
    // mutable reference, and so there must currently be no existing  
    // references. Thus, while clone increments the mutable refcount, here  
    // we explicitly only allow going from UNUSED to UNUSED - 1.  
    match borrow.get() {  
      UNUSED => {  
        borrow.set(UNUSED - 1);  
        Some(BorrowRefMut { borrow })  
      }  
      _ => None,  
    }  
  }  
}
```

BorrowRef vs BorrowRefMut

```
impl<'b> BorrowRef<'b> {  
  
    fn new(borrow: &'b Cell<BorrowFlag>) ->  
        Option<BorrowRef<'b>> {  
  
        let b = borrow.get().wrapping_add(1);  
        if !is_reading(b) {  
            None  
        } else {  
            borrow.set(b);  
            Some(BorrowRef { borrow })  
        }  
    }  
}  
  
fn is_reading(x: BorrowFlag) -> bool {  
    x > UNUSED  
}
```

```
impl<'b> BorrowRefMut<'b> {  
  
    fn new(borrow: &'b Cell<BorrowFlag>) ->  
        Option<BorrowRefMut<'b>> {  
  
        match borrow.get() {  
            UNUSED => {  
                borrow.set(UNUSED - 1);  
                Some(BorrowRefMut { borrow })  
            }  
            _ => None,  
        }  
    }  
}
```

RefMut

```
pub struct RefMut<'b, T: ?Sized + 'b> {  
    value: NonNull<T>,  
    borrow: BorrowRefMut<'b>,  
    marker: PhantomData<&'b mut T>,  
}
```

Homework

- Rustlings
 - smart_pointers
 - vecs
 - Strings
- Courseware
 - New feature: sorting. Key --sort: output should be sorted by file name
 - cargo run -- c:/Projects --find main.rs --sort
 - > abra.rs
 - > folder/cadabra.rs
 - > shwabra.txt
 - Save folders/files to memory
 - Sorting algorithm: implement on your own